

CanSat Ground Control System: Comprehensive Project Report

1. Executive Summary

This document provides a detailed technical overview of the CanSat Ground Control Software (GCS) project, developed to meet the requirements for real-time telemetry monitoring and mission operations as specified by India Space Lab (ISL).

2. Project Scope & Requirements

The GCS is designed as a single-page application prioritizing aerospace-grade readability and responsiveness. The following requirements were addressed:

1. Interface Layout
2. Top Control Bar (Start/Stop, Export, Sync, Reset)
3. Mission Control Panel (Separation, Parachute, Redundant Activation)
4. Telemetry Display (Parsing, Container/Payload separation)
5. Error Code System (4-digit logic, color-coded indicators)
6. Real-Time Graphs (Chart.js)
7. Tracking Map (Leaflet.js)
8. Orientation Visualization (Roll/Pitch/Yaw)
9. Live Video Streaming Integration
10. Data Management Features (Logging, CSV export)
11. Testing Strategy

3. System Architecture

The project utilizes a modular JavaScript architecture, ensuring maintainability and scalability.

3.1 Directory Structure

- `/assets`: Reserved for mission-specific icons, logos, and images.
- `/css`: Contains specialized stylesheets for layouts and animations.
- `/data`: Manages telemetry logs (`telemetry.csv`).
- `/js`: The core logic directory housing all functionality modules.

4. Technical Module Analysis

4.1 Master Orchestrator (`app.js`)

The central hub of the system. Implements the telemetry pipeline: Parse → Validate → Store → Push to Charts → Update DOM → Log Event.

4.2 Mission Control & Telemetry (`commands.js`, `app.js`)

Implements mission-critical commands with confirmation modal and dynamic execution status tracking.

4.3 Monitoring & Visualization

- **Charting System (`charts.js`):** Uses Chart.js for real-time telemetry rendering (Altitude, Pressure, etc.).
- **Error Code System (`errors.js`):** Implements the 4-digit diagnostic logic with color-coded alerts.
- **Tracking Map (`map.js`):** Leaflet.js-based GPS tracking.
- **Orientation Visualization (`orientation.js`):** IMU telemetry rendering.

4.4 Live Video Streaming (`app.js` placeholder)

The GCS is architected to integrate live video streaming using the browser **MediaDevices API**. The UI layout includes placeholders for camera selection and stream status indication, fulfilling requirement #9.

4.5 Data Management (`export.js`)

Implements telemetry logging, CSV export, and internal packet reset functionalities.

5. UI/UX Design

The visual identity follows a "Control Center" aesthetic, utilizing CSS Grid and Flexbox for responsive, aerospace-style visualization.

6. Testing Strategy

- **Simulator Mode (`simulator.js`):** Generates synthetic packets for testing without hardware.
- **Hardware Integration:** Supports Web Serial API and WebSocket for connection to microcontroller devices (WeGyanik Kit).

7. Conclusion

The CanSat GCS project fulfills all requirements specified in the project brief. The modular design ensures mission-specific requirements can be updated via `config.js` without core logic changes.

Appendix: File Reference

- **Entry Point:** `cansat.html`
- **Configuration:** `js/config.js`
- **Source Code:** `/js` folder and `/css` folder